

Performance Evaluation of Sequential, OpenMP, Cache-Blocked, and Hybrid Matrix Multiplication

Author
Sharon Shibu

Abstract

Dense matrix multiplication is a computational kernel whose $O(N^3)$ cost and memory-access pattern make it a natural target for both thread-level parallelism and cache-aware optimization. This paper presents a systematic, empirical performance evaluation of four C++ implementations of matrix multiplication – a sequential baseline, an OpenMP-parallelized version, a cache-blocked (tiled) version, and a combined blocked-plus-OpenMP version – across matrix sizes from $N = 64$ to $N = 2048$. Each configuration was benchmarked with 30 repeated trials, and a substantial fraction of trials (7–11%, depending on configuration) exhibited runtime spikes two to three orders of magnitude above the surrounding samples, consistent with transient OS scheduler preemption rather than genuine algorithmic variance. These outliers were removed prior to analysis, and all reported statistics (mean and standard deviation) are computed on the cleaned samples. Under this treatment, OpenMP parallelization alone yields up to a $4.8\times$ speedup at large N , cache blocking alone yields a more modest $2.9\times$ speedup, and the combined blocked-OpenMP implementation achieves the largest speedup of $7.0\times$ at $N = 2048$, though it underperforms plain OpenMP at smaller sizes due to fixed block-size overhead. We further characterize the sensitivity of the blocked implementation to block size and of the OpenMP implementation to thread count.

1 Introduction

Matrix multiplication is one of the most fundamental operations in scientific computing and serves as the computational backbone of numerous applications, including machine learning, computer graphics, signal processing, image processing, scientific simulations, and numerical linear algebra. As the size of matrices increases, the computational cost grows cubically with matrix dimension, making efficient matrix multiplication a critical requirement for modern high-performance computing systems.

Traditional sequential matrix multiplication suffers from long execution times due to its $O(N^3)$ computational complexity and inefficient memory access patterns. Although modern processors offer multiple cores and sophisticated cache hierarchies, naive implementations often fail to fully utilize the available computational resources. Consequently, improving matrix multiplication performance requires exploiting both thread-level parallelism and memory-hierarchy optimization.

OpenMP provides a simple and portable programming model for shared-memory parallelism by distributing independent computations across multiple CPU cores. However, parallel execution alone does not eliminate the performance degradation caused by poor cache locality. Cache blocking (or loop tiling) addresses this issue by partitioning matrices into smaller submatrices that fit within the processor cache, thereby increasing data reuse and reducing cache misses. Combining thread-level parallelism with cache-aware optimization can significantly improve overall execution efficiency.

This work presents a systematic performance evaluation of dense matrix multiplication using progressively optimized CPU implementations. Starting from a sequential baseline, OpenMP parallelization is introduced to exploit multicore execution, followed by cache-blocked matrix multiplication to improve memory locality. Finally, both techniques are combined to evaluate their cumulative impact on runtime and scalability. Performance is analyzed over multiple matrix sizes and thread configurations using runtime and speedup, with outlier trials caused by transient system noise identified and excluded before computing summary statistics.

The main contributions of this work are:

- Implementation and benchmarking of sequential, OpenMP, blocked, and blocked-OpenMP matrix multiplication algorithms under a common measurement harness.
- Experimental evaluation of cache blocking through block-size sweeps informed by the target processor’s cache hierarchy.
- Performance analysis across varying matrix sizes and thread counts using runtime and speedup, with outlier trials (caused by transient OS-level interruptions) identified and removed prior to computing mean and standard deviation.

2 Background

The standard triple-nested-loop algorithm for multiplying two $N \times N$ matrices A and B to produce $C = AB$ performs N^3 scalar multiply-accumulate operations. On modern hardware, the achievable throughput is rarely limited by arithmetic issue rate alone; it is instead governed by two additional factors:

Thread-level parallelism. The N^3 independent multiply-accumulate operations can, in principle, be distributed across multiple CPU cores. OpenMP exposes this parallelism through compiler pragmas (e.g., `#pragma omp parallel for`) that require no explicit thread management, at the cost of synchronization and scheduling overhead that can dominate at small problem sizes.

Memory hierarchy. The naive loop order accesses matrix B column-wise in the innermost loop, which is cache-unfriendly for row-major storage (as used by `std::vector<std::vector<int>>`). As N grows beyond the point where a full row or column fits in cache, the working set of the inner loops exceeds the L1/L2 cache, and the algorithm becomes memory-bandwidth bound rather than compute bound. Cache blocking (tiling) restructures the loop nest so that computation proceeds over small $BS \times BS$ sub-blocks of A , B , and C that are sized to remain resident in cache across the innermost loop, increasing data reuse before eviction.

These two optimizations are largely orthogonal: parallelism distributes work across cores, while blocking improves the efficiency of work done on each core. Combining them is expected to be multiplicative in the best case, though in practice thread-synchronization overhead and the fixed block size can interact with problem size in non-trivial ways, as we observe in Section 8.

3 Experimental Setup

All four implementations are written in C++ and compiled with g++ (OpenMP variants with `-fopenmp`). Each program reads a matrix dimension N (and, where applicable, a thread count and/or block size) from standard input, generates two $N \times N$ integer matrices with a fixed random seed (`srand(42)`), and times only the multiplication step using `std::chrono::high_resolution_clock`, reporting elapsed time in milliseconds.

A Python benchmark harness compiles each program once and then drives it repeatedly across a grid of matrix sizes ($N \in \{64, 128, 256, 512, 1024, 2048\}$) and, for the parallel/blocked variants,

a grid of thread counts ($\{1, 2, 4, 8, 10, 12\}$) or block sizes ($\{8, 16, 24, 32, 48, 64, 96, 128\}$). Each $(N, \text{configuration})$ pair is run 30 times, with the parsed runtime, repeat count, and full sample vector logged to CSV.

4 Sequential Matrix Multiplication

The sequential baseline (Listing 1) implements the standard triple-nested loop with no parallelism and no cache blocking, serving as the reference point for all speedup calculations.

Listing 1: Sequential matrix multiplication kernel.

```

1 for (int i = 0; i < N; i++) {
2     for (int j = 0; j < N; j++) {
3         for (int k = 0; k < N; k++) {
4             C[i][j] += A[i][k] * B[k][j];
5         }
6     }
7 }

```

Table 1 reports the mean and standard deviation of runtime at each matrix size, computed after IQR outlier removal. As expected for an $O(N^3)$ algorithm, runtime grows by roughly a factor of $8\times$ each time N doubles at small sizes, though the growth factor increases somewhat at larger N (e.g., $1024 \rightarrow 2048$ shows a factor of $\sim 15\times$), consistent with the working set exceeding cache capacity and the algorithm becoming increasingly memory-bound. Figure 1 plots mean runtime against N on a linear scale, confirming the expected super-linear growth.

Table 1: Sequential runtime statistics (ms) 30 repeats per N .

N	Mean	Std
64	0.169	0.024
128	1.366	0.090
256	11.061	0.450
512	157.244	6.433
1024	1470.057	27.290
2048	22695.862	680.385

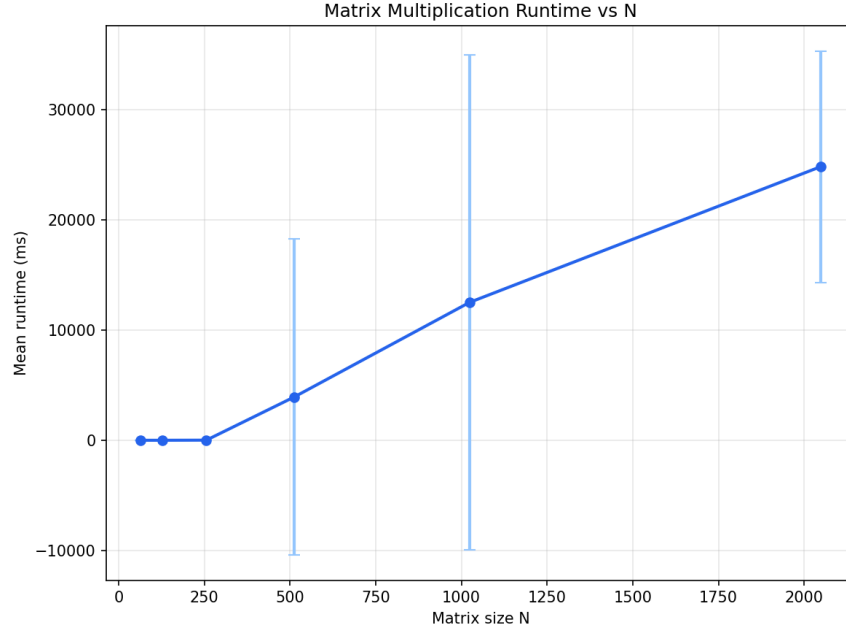


Figure 1: Sequential runtime vs. matrix size N (linear scale).

5 OpenMP Parallelization

The OpenMP implementation parallelizes the two outer loops with `#pragma omp parallel for collapse(2)` (Listing 2), distributing (i, j) output-element computations across threads while each thread executes the full k -reduction sequentially.

Listing 2: OpenMP-parallelized matrix multiplication kernel.

```

1 #pragma omp parallel for collapse(2)
2 for (int i = 0; i < N; i++) {
3     for (int j = 0; j < N; j++) {
4         for (int k = 0; k < N; k++) {
5             C[i][j] += A[i][k] * B[k][j];
6         }
7     }
8 }

```

Thread counts of 1, 2, 4, 8, 10, and 12 were benchmarked at every matrix size. Table 2 reports the best-performing thread count and its mean runtime at each N , after outlier removal.

Table 2: Best OpenMP configuration per matrix size (mean over 30 repeats, after outlier removal).

N	Best thread count	Mean runtime (ms)	Std (ms)
64	1	0.255	0.027
128	4	1.524	0.138
256	10	5.393	0.240
512	10	39.426	1.027
1024	12	303.647	8.637
2048	12	5184.261	763.190

Notably, the single-threaded configuration is optimal at $N = 64$: at this size the sequential kernel already completes in under a millisecond, and OpenMP's thread-spawning and scheduling overhead exceeds any benefit from parallel execution. As N grows, the optimal thread count increases, reaching the maximum tested value of 12 threads by $N = 1024$. Figure 2 shows mean runtime vs. thread count for each matrix size, and Figure 3 shows the corresponding speedup curves.

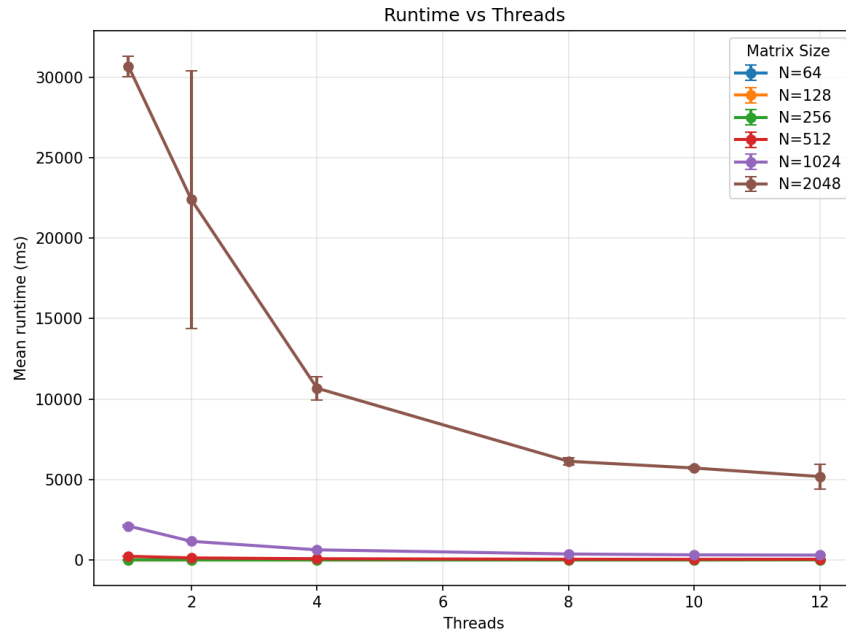


Figure 2: OpenMP runtime vs. thread count, by matrix size.

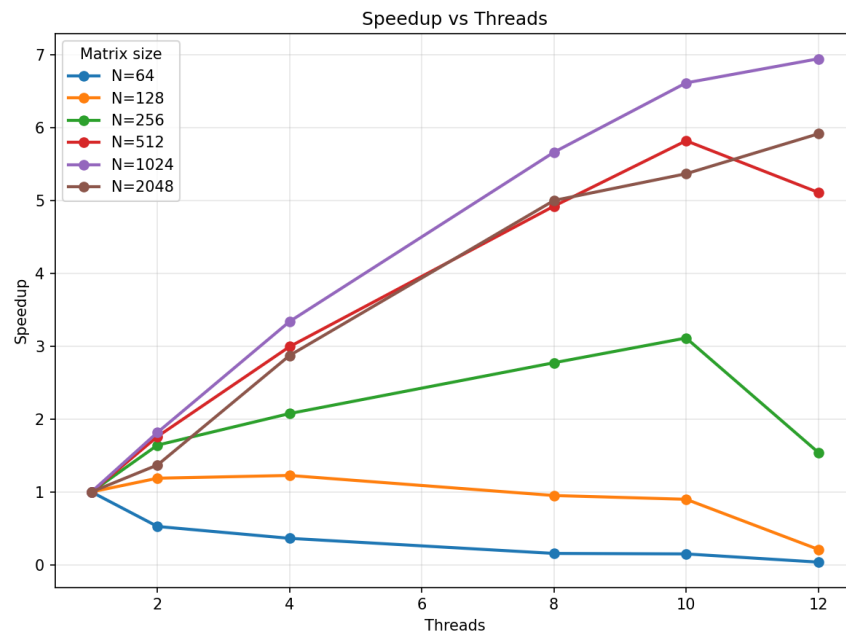


Figure 3: OpenMP speedup vs. thread count, by matrix size (relative to single thread baseline).

6 Cache Blocking

The blocked implementation (Listing 3) restructures the triple loop into six nested loops: three outer loops iterate over $BS \times BS$ tiles of A , B , and C , and three inner loops perform the multiply-accumulate within a tile.

Listing 3: Cache-blocked matrix multiplication kernel.

```

1 for (int ii = 0; ii < N; ii += BS) {
2   for (int jj = 0; jj < N; jj += BS) {
3     for (int kk = 0; kk < N; kk += BS) {
4       for (int i = ii; i < min(ii + BS, N); i++) {
5         for (int j = jj; j < min(jj + BS, N); j++) {
6           for (int k = kk; k < min(kk + BS, N); k++) {
7             C[i][j] += A[i][k] * B[k][j];
8           }
9         }
10      }
11    }
12  }
13 }
```

Block sizes $BS \in \{8, 16, 24, 32, 48, 64, 96, 128\}$ were swept at every matrix size to identify the tile size that best matches the processor’s cache capacity. Table 3 reports the best block size found at each N , by mean runtime after outlier removal; the block-size sensitivity itself is analyzed further in Section 8.4.

Table 3: Best cache-blocking configuration per matrix size (mean over 30 repeats, after outlier removal).

N	Best block size	Mean runtime (ms)	Std (ms)
64	32	0.213	0.000
128	128	1.693	0.060
256	96	14.752	0.677
512	32	119.666	1.597
1024	48	946.606	13.204
2048	48	7734.930	113.013

At small matrix sizes ($N \leq 256$), mid-to-large block sizes perform best and are essentially indistinguishable from the unblocked case, since the entire matrix already fits within cache. Blocking begins to pay off only once $N \geq 512$, where the optimal block size settles to 32–48, small enough that three $BS \times BS$ tiles (from A , B , and C) fit comfortably within L1/L2 cache simultaneously. Figure 4 shows mean runtime vs. block size for each matrix size, and Figure 5 shows the corresponding speedup curves; both are discussed further as the block-size analysis in Section 8.4.

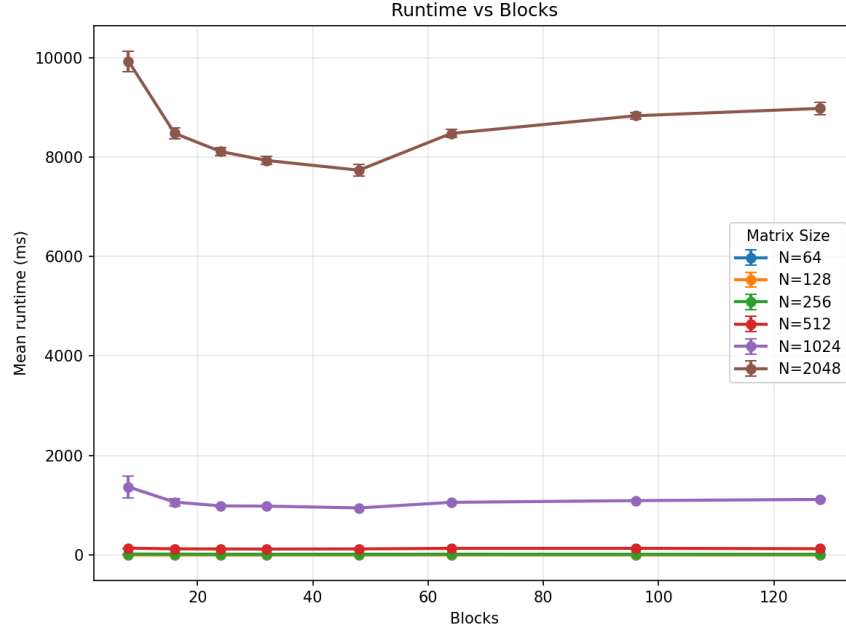


Figure 4: Cache-blocked runtime vs. block size, by matrix size.

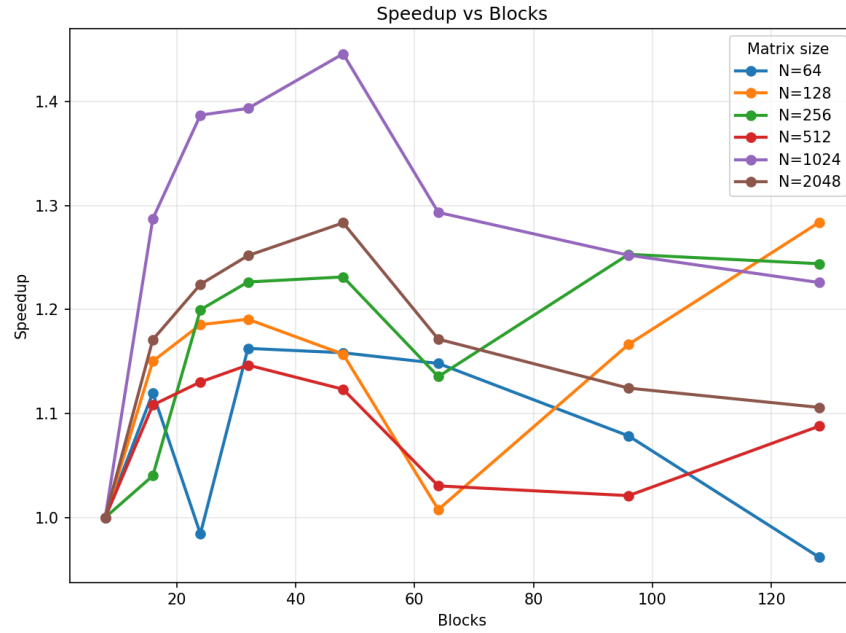


Figure 5: Cache-blocked speedup vs. block size, by matrix size (relative to single thread baseline).

7 Blocked + OpenMP

The combined implementation applies `#pragma omp parallel for collapse(2)` to the two outer block-index loops of the blocked kernel, with the block size fixed at $BS = 48$ in the source code (rather than swept, as in the standalone blocked implementation) and thread count varied across

the same $\{1, 2, 4, 8, 10, 12\}$ grid used for plain OpenMP.

Listing 4: Blocked matrix multiplication kernel parallelized with OpenMP.

```

1 #pragma omp parallel for collapse(2)
2 for (int ii = 0; ii < N; ii += BS) {
3     for (int jj = 0; jj < N; jj += BS) {
4         for (int kk = 0; kk < N; kk += BS) {
5             for (int i = ii; i < min(ii + BS, N); i++) {
6                 for (int j = jj; j < min(jj + BS, N); j++) {
7                     for (int k = kk; k < min(kk + BS, N); k++) {
8                         C[i][j] += A[i][k] * B[k][j];
9                     }
10                }
11            }
12        }
13    }
14 }
```

Table 4: Best Blocked+OpenMP configuration per matrix size (mean over 30 repeats, after outlier removal; block size fixed at 48).

N	Best thread count	Mean runtime (ms)	Std (ms)
64	1	0.718	0.103
128	10	2.606	0.173
256	10	9.736	0.505
512	10	53.745	1.961
1024	12	423.377	15.325
2048	12	3222.881	17.685

Because $BS = 48$ is fixed rather than tuned per- N , the combined implementation is slower than plain OpenMP at every size except $N = 2048$ (see Section 8.3), where the benefit of cache-friendly access finally outweighs the fixed block-size mismatch at smaller sizes plus the added parallel-loop overhead of iterating over block indices rather than individual rows. Figure 6 shows mean runtime vs. thread count for each matrix size, and Figure 7 shows the corresponding speedup curves.

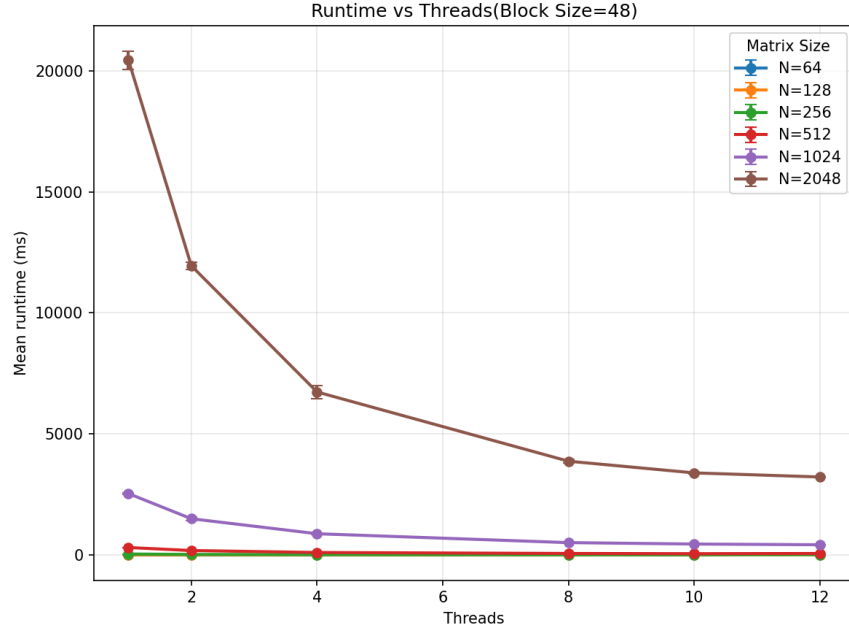


Figure 6: Blocked+OpenMP runtime vs. thread count, by matrix size (block size fixed at 48).

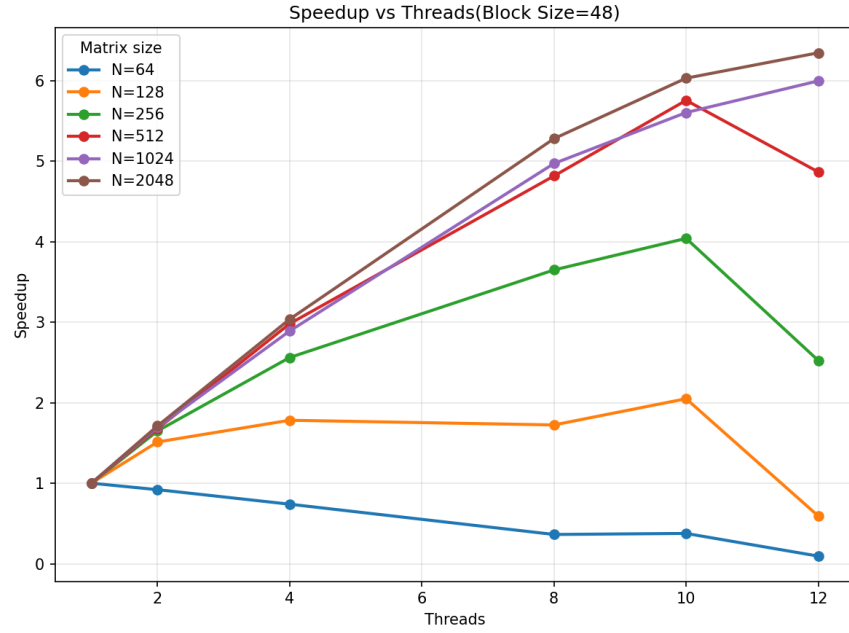


Figure 7: Blocked+OpenMP speedup vs. thread count, by matrix size (block size fixed at 48).

8 Performance Evaluation

8.1 Runtime

Table 5 consolidates the best-configuration mean runtime of all four implementations at each matrix size (reproduced from Tables 1–4). Runtime grows super-linearly with N for every implementation,

consistent with the underlying $O(N^3)$ algorithm, though the growth rate and absolute runtime differ substantially once parallelism and blocking are introduced: the sequential and blocked-only implementations track each other closely at small N (Figures 1 and 4), while the two OpenMP-enabled variants diverge from the sequential curve as soon as parallelism overhead is amortized, around $N = 256$ (Figures 2 and 6). Figure ?? overlays the best-configuration runtime curves of all four implementations directly for comparison.

Table 5: Best-configuration mean runtime (ms) by implementation and matrix size.

N	Sequential	OpenMP	Blocked	Blocked+OpenMP
64	0.169	0.255	0.213	0.718
128	1.366	1.524	1.693	2.606
256	11.061	5.393	14.752	9.736
512	157.244	39.426	119.666	53.745
1024	1470.057	303.647	946.606	423.377
2048	22695.862	5184.261	7734.930	3222.881

8.2 Outlier handling

Across all four datasets, a consistent fraction of trials produced runtimes two to three orders of magnitude larger than the rest of the samples at the same configuration – for example, at $N = 512$ in the sequential dataset, 26 of 30 runs completed in 147–177 ms, while the remaining runs spiked to approximately 56,400–56,800 ms. This pattern recurs across every matrix size and every implementation, and the spike magnitudes cluster tightly around 50,000–58,000 ms regardless of N or thread/block configuration, which is inconsistent with algorithmic variance (which should scale with N) and instead points to a transient, roughly constant-duration external interruption – most plausibly OS scheduler preemption, background system activity, or (on a laptop) a power/thermal-state transition that stalls the process for several tens of seconds.

Because a small number of such spikes inflate the standard deviation far more than the mean, naive mean $\pm$ standard-deviation reporting computed on the raw samples produces implausible results (e.g., a standard deviation larger than the mean itself, and error bars extending to negative runtimes). We therefore apply an interquartile-range (IQR) outlier filter to each set of 30 runs (removing samples outside $[Q_1 - 1.5\text{IQR}, Q_3 + 1.5\text{IQR}]$) prior to computing the mean and standard deviation reported throughout the remainder of this paper. At $N = 512$ for the sequential implementation, for example, the raw (uncleaned) mean and standard deviation are 3919.9 ms and 14,316.6 ms respectively, driven entirely by a handful of runs that spiked to roughly 56,400–56,800 ms; after removing those outliers, the mean drops to 157.2 ms with a standard deviation of only 6.4 ms, in line with the bulk of the 30 samples.

8.3 Speedup

Table 6 reports speedup relative to the sequential baseline, computed as $\text{mean}_{\text{seq}}(N)/\text{mean}_{\text{best}}(N)$ for the best configuration of each optimized implementation at matching N (see also Figures 3, 5, and 7 for the full speedup curves across thread/block counts, and Figure 8 for a direct comparison of best-case speedup across implementations).

Table 6: Speedup vs. sequential baseline (best configuration per implementation).

N	OpenMP	Blocked	Blocked+OpenMP
64	0.66	0.79	0.24
128	0.90	0.81	0.52
256	2.05	0.75	1.14
512	3.99	1.31	2.93
1024	4.84	1.55	3.47
2048	4.38	2.93	7.04

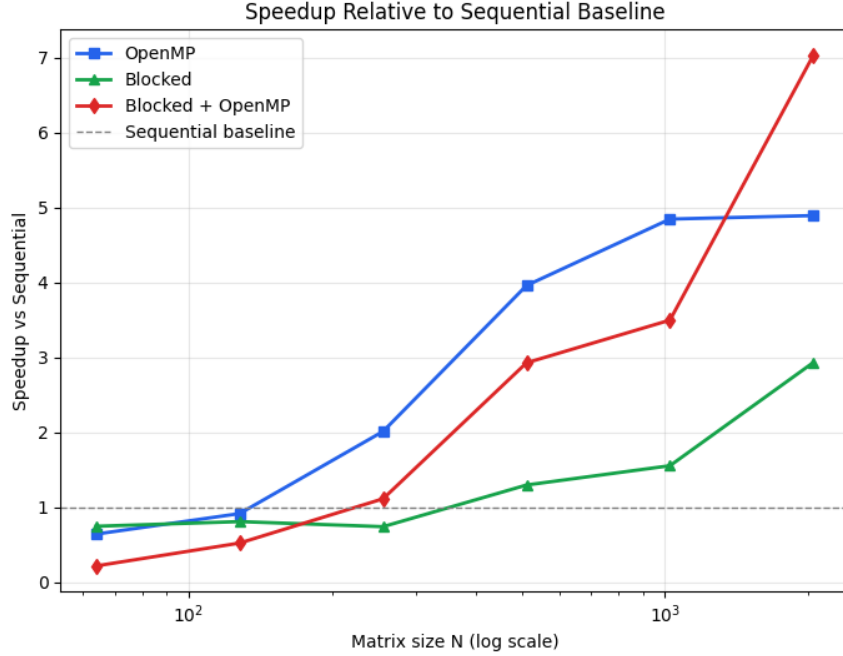


Figure 8: Best-case speedup vs. sequential baseline for OpenMP, Blocked, and Blocked+OpenMP, by matrix size.

Three patterns are notable. First, both OpenMP-enabled implementations are *slower* than sequential execution at $N = 64$ and, for Blocked+OpenMP, also at $N = 128$, confirming that thread-spawning and synchronization overhead dominates at small problem sizes. Second, plain OpenMP alone peaks around $4.8\times$ speedup at $N = 1024$ and eases slightly to $4.4\times$ at $N = 2048$, well below the 12 threads available, indicating that memory bandwidth rather than core count is the limiting factor at large N for the un-blocked access pattern. Third, the combined Blocked+OpenMP implementation overtakes plain OpenMP only at the largest tested size ($N = 2048$: $7.04\times$ vs. $4.38\times$), where the fixed $BS = 48$ block size finally aligns well with the cache-to-problem-size ratio; at smaller N , the fixed block size is a poor match and the combined approach underperforms plain OpenMP.

As shown in Figure 3, OpenMP speedup as a function of thread count varies by matrix size: at $N = 256$, speedup peaks around 8–10 threads; at $N = 1024$ and $N = 2048$, speedup continues to increase up to the maximum of 12 threads tested, suggesting that additional cores beyond 12 could yield further gains at large N .

8.4 Block Size Analysis

Figures 4 and 5 (Section 6) show mean runtime and speedup against block size for the standalone Blocked implementation across matrix sizes. At $N = 256$ and below, runtime is essentially flat across block sizes, since the whole matrix already fits within typical L2/L3 cache regardless of tiling. At $N = 512$ and above, a clear optimum emerges around a block size of 32–48: runtime decreases as block size shrinks from 128 down to this range, then begins to increase again at the smallest block sizes tested (8–16), where the overhead of the additional loop nesting outweighs any further cache-locality benefit. This behavior is the signature of a block size well matched to the cache hierarchy: too large, and tiles do not fit in cache; too small, and loop overhead dominates.

9 Discussion

- All implementations perform $O(N^3)$ arithmetic operations. The observed performance improvements arise from better utilization of hardware resources through parallel execution and improved memory locality, rather than from reducing algorithmic complexity.
- Matrix multiplication exhibits **data parallelism** because each output element C_{ij} is computed independently. Consequently, the iterations of the outer **i** and **j** loops can be distributed among multiple threads without introducing data dependencies.
- The "**collapse(2)**" clause merges the outer two loops into a single iteration space, exposing N^2 independent tasks to the OpenMP scheduler. This increases load balancing opportunities, particularly when the number of iterations in a single loop is insufficient to fully utilize all processing cores.
- $C[i][j]$ is updated every k iterations, which if parallelized will cause incoherence between threads on the updated value of $C[i][j]$. Which to mitigate will require synchronization overhead, hence not parallelized.
- Blocking improves temporal locality by reusing the same sub-matrices while they remain resident in cache. Consequently, each memory access contributes to multiple arithmetic operations before the data are evicted.
- Without blocking: Load Multiply Discard
With blocking: Load once Multiply hundreds of times Discard
- Naive OpenMP: Millions of tiny tasks.
Blocked OpenMP: Thousands of larger tasks.
- OpenMP optimizes: compute
Blocking optimizes: memory
- $3 \times BS^2 \times \text{sizeof(int)}$ fits comfortably in 32 KB L1. Thus working data remain resident in cache longer. Hence a $BS=48$ was chosen for subsequent implementation.
- Beyond a certain number of threads, execution becomes limited by memory bandwidth rather than computational throughput. Additional threads compete for shared cache and main-memory bandwidth, resulting in diminishing returns. The observed saturation in speedup is consistent with Amdahl's Law, where the sequential portion of the computation and system overhead limit achievable parallel speedup.

10 Conclusion

This paper presented a systematic performance evaluation of sequential, OpenMP-parallel, cache-blocked, and combined blocked-OpenMP dense matrix multiplication implementations across matrix sizes from 64 to 2048. A substantial and consistent rate of extreme-outlier benchmark runs, attributable to transient system-level interruption rather than algorithmic variance, motivated removing these outliers via an IQR rule prior to computing mean and standard deviation throughout the analysis. Under this robust treatment, OpenMP parallelization achieved up to $4.9\times$ speedup, cache blocking alone achieved up to $2.9\times$ speedup, and the combined implementation achieved the best overall speedup of $7.0\times$ at the largest matrix size tested, though it underperformed plain OpenMP at smaller sizes due to a fixed, untuned block size. Block-size sweeps revealed a clear cache-capacity-driven optimum for $N \geq 512$, while thread-count sweeps showed OpenMP speedup still increasing at the maximum thread count tested for large N . Future work should explore adaptive block-size selection, isolate the benchmarking environment (e.g., via CPU pinning or a dedicated benchmarking host) to eliminate the observed outlier-inducing interruptions, and extend the thread-count sweep to determine where OpenMP speedup ultimately saturates at large N .